

AI-BASED ONLINE PROCTORING SYSTEM

[Risk synthesis Agent]

1. Name

Project Name: AI-Based Online Examination Proctoring System using LangGraph, Computer Vision, Audio Analysis and RAG

Primary Objective: To develop a modular AI-assisted proctoring pipeline that analyzes examination video and audio, checks the evidence against examination rules, assesses risk, and produces a human-review recommendation.

2. Implementation

Step 1 – Project Setup and Folder Structure

Create the project structure separating the LangGraph workflow, agents, video processing, computer vision, RAG, decision engine, LLM, and data directories. The shared workflow state is maintained in `graph/state.py` and the LangGraph workflow is defined in `graph/workflow.py`.

Step 2 – Input Video

Place the examination recording in `data/videos/input_video.mp4`. The video is used as the primary source for visual and audio analysis.

Step 3 – Video and Frame Processing

Read the examination video and extract representative frames into `data/frames/`. The frames provide the visual evidence used by the proctoring pipeline.

Step 4 – Visual/Object Analysis

Analyze extracted frames for relevant objects and visual events such as persons, mobile phones or other potentially unauthorized objects. The existing `vision/object_detector.py` module is retained for the project's computer-vision processing.

Step 5 – Audio Extraction

Extract the audio track from the input video using FFmpeg and convert it to a suitable format such as mono 16 kHz WAV. The generated file is stored in `data/audio/audio.wav`.

Step 6 – Speech-to-Text

Use Whisper to convert examination audio into a text transcript. The audio agent uses FP32 inference to avoid the CUDA/FP16 NaN-logit issue encountered during Whisper transcription, with CPU fallback when CUDA inference fails.

Step 7 – RAG Document Preparation

Load the examination rules from `data/rules/rules.txt`. The rules are divided into manageable chunks using the RAG chunking component so that relevant rule information can be retrieved for each analysis.

Step 8 – Embeddings and Retrieval

Generate embeddings for the rule chunks and retrieve the rules relevant to the observed evidence. This allows the system to ground risk analysis in the examination policy rather than relying only on general model knowledge.

Step 9 – Behavior Analysis

Combine visual observations and audio/transcript information to identify potentially suspicious behavioral patterns. The behavior agent records the available observations in the shared LangGraph state.

Step 10 – Rule Evaluation and Risk Analysis

Compare the available evidence with the retrieved examination rules. The risk agent determines a risk level such as LOW, MEDIUM, or HIGH and records the supporting risk analysis.

Step 11 – LLM-Based Evidence Synthesis

Pass the visual evidence, transcript, behavioral analysis, retrieved rules, and risk analysis to the proctoring LLM. The synthesis agent creates an evidence-based summary while distinguishing observations from assumptions.

Step 12 – Human Review Recommendation

Generate a human-review recommendation based on the risk analysis and evidence synthesis. The system identifies what a human proctor should inspect and assigns a recommendation priority.

Step 13 – LangGraph Orchestration

Use LangGraph to orchestrate the complete workflow. The sequence is START → Video Agent → Audio Agent → Behavior Agent → Risk Agent → Synthesis Agent → Human Review Agent → END.

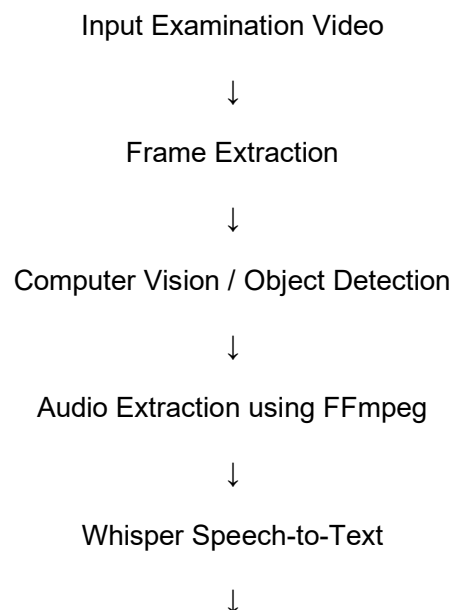
Step 14 – Final Report Generation

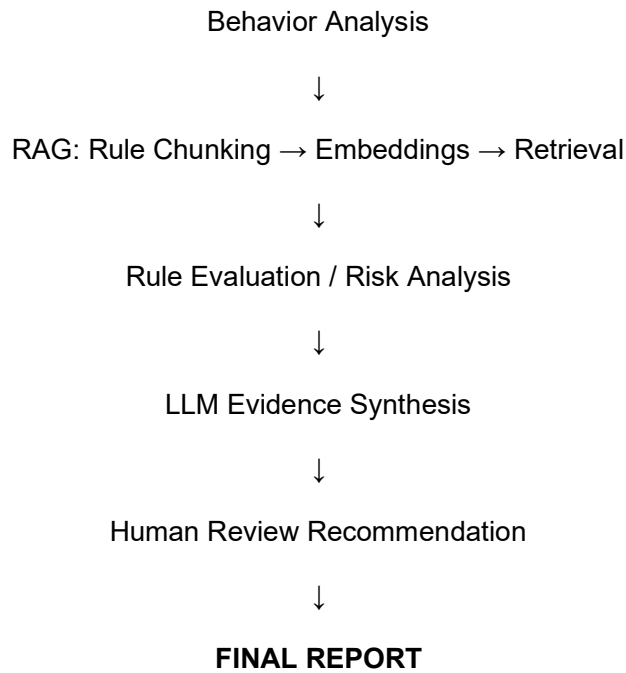
Collect the outputs from all agents and generate a final report containing the risk analysis, human-review recommendation, risk level, recommendation priority, evidence summary, and step history.

Step 15 – Validation and Testing

Run the complete workflow using sample examination videos and verify that frames, audio, transcript, rule retrieval, risk analysis, human review, and final report are produced correctly.

3. Implementation Architecture





4. Scope of Work

Video Analysis: Process examination recordings and extract frames for visual inspection.

Computer Vision: Identify relevant visual objects and observations from examination frames.

Audio Analysis: Extract audio and convert speech into text using Whisper.

Behavior Analysis: Analyze available visual and audio evidence for potentially suspicious patterns.

RAG-Based Rule Grounding: Use examination rules as the knowledge source for evidence-based evaluation.

Risk Assessment: Classify observed risk as LOW, MEDIUM, or HIGH based on the available evidence.

LLM Synthesis: Generate a concise evidence-based synthesis of the examination findings.

Human-in-the-Loop Review: Provide recommendations for a human reviewer rather than automatically declaring cheating.

LangGraph Workflow: Coordinate individual agents through a shared state and controlled execution flow.

Final Reporting: Produce a structured final report containing findings, risk level, review recommendation, priority, and processing history.

5. Main Project Components

Component	Purpose
main.py	Starts the LangGraph workflow and prints the final report.
graph/state.py	Defines the shared state passed between agents.
graph/workflow.py	Defines and compiles the LangGraph workflow.

agents/video_agent.py	Handles video/frame analysis.
agents/audio_agent.py	Extracts audio and performs Whisper transcription.
agents/behavior_agent.py	Produces behavioral observations.
agents/risk_agent.py	Evaluates evidence and determines risk.
agents/synthesis_agent.py	Creates the evidence synthesis.
agents/human_review_agent.py	Generates human-review recommendations.
video_processing/	Existing video and frame-processing modules.
vision/	Existing computer-vision/object-detection modules.
rag/	Chunking, embedding, and retrieval modules for examination rules.
decision/	Existing rule-engine/decision modules.
llm/proctoring_llm.py	LLM interaction and report-generation functions.
data/rules/rules.txt	Source examination rules used by the RAG/rule-analysis stage.

6. Expected Output

The completed system should generate a structured final report containing:

- Risk analysis
- Human review recommendation
- Risk level
- Recommendation priority
- Video evidence summary
- Audio transcript, when available
- Rules considered
- Step history showing which agents completed

Important: The system is designed as an AI-assisted decision-support system. A high-risk result should trigger human investigation and should not, by itself, be treated as conclusive proof of cheating.

Further Planning

1. Transition from Rule-Based to AI-Based Decision Making

The current system uses rules.txt and a rule-based decision layer to determine whether detected events violate examination rules. In the next phase, we plan to remove the dependency on rules.txt and the traditional rule-based approach. The system will instead use AI/LLM-based reasoning to interpret detected visual, audio, and behavioral evidence and determine the potential severity and relevance of each event. This will make the system more dynamic and reduce dependence on manually defined rules.

2. Fully AI-Driven Risk Assessment

The future system will combine outputs from the Video Agent, Audio Agent, and Behavior Agent and use an AI-based reasoning layer for contextual analysis. Rather than checking only predefined conditions, the AI will consider multiple pieces of evidence together before generating the final risk assessment.

3. Code Optimization

The existing implementation will be optimized to reduce unnecessary processing, improve execution speed, minimize memory/GPU usage, and make the overall LangGraph workflow more efficient. Model

loading, frame processing, audio transcription, LLM calls, and data passing between agents will be reviewed and optimized.

4. Bug Fixing and System Stability

Known bugs and runtime issues will be identified and fixed. This includes handling model errors, GPU-related issues, invalid outputs, missing files, unexpected agent failures, and workflow/state-management problems. Proper error handling and logging will also be added to make the system more stable.

5. User Interface Development

A dedicated UI will be developed for the proctoring system. The interface will allow users/reviewers to upload examination videos and view the processing status, detected suspicious events, timestamps, audio findings, behavioral analysis, risk level, evidence, and final human-review recommendations in a clear and user-friendly dashboard.

6. End-to-End AI Proctoring Platform

The final objective is to transform the current prototype into a more optimized, stable, and AI-driven proctoring platform where video, audio, and behavioral evidence are analyzed together, AI performs contextual reasoning, and the results are presented through an intuitive UI for human review.